This CVPR paper is the Open Access version, provided by the Computer Vision Foundation.
Except for this watermark, it is identical to the accepted version;
the final published version of the proceedings is available on IEEE Xplore.

DOF-GS: Adjustable Depth-of-Field 3D Gaussian Splatting for Post-Capture Refocusing, Defocus Rendering and Blur Removal

Yujie Wang[1,2,3] Praneeth Chakravarthula[3†] Baoquan Chen[1,2†]

[1]State Key Laboratory of General Artificial Intelligence, Peking University
[2]School of Intelligence Science and Technology, Peking University
[3]University of North Carolina at Chapel Hill

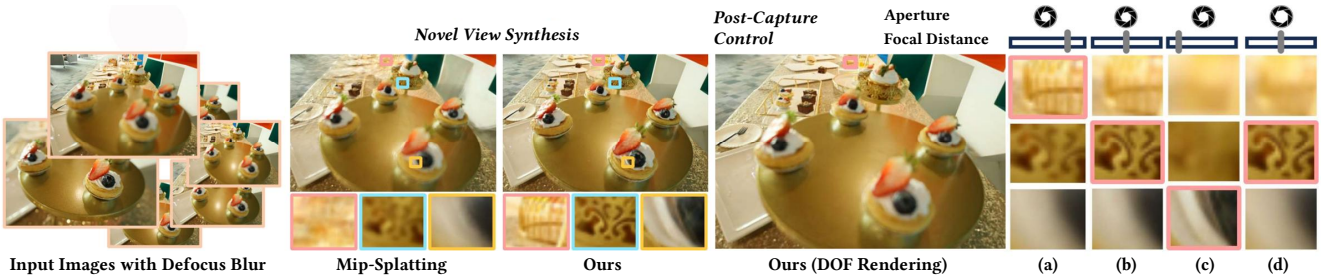


Figure 1. Our method enhances 3DGS with a finite-aperture camera model and differentiable depth-of-field (DOF) rendering, enabling post-capture DOF control after training and capturing natural defocus cues from multi-view defocused images. Differentiable DOF rendering also mitigates reconstruction artifacts caused by multi-view inconsistent defocus blur. Insets (a)-(d), with in-focus areas in pink, show images rendered under various settings: (a)-(c) share aperture but differ in focal distance; (d) matches (b)'s focal distance but has a larger aperture.

Abstract

3D Gaussian Splatting (3DGS) techniques have recently enabled high-quality 3D scene reconstruction and real-time novel view synthesis. These approaches, however, are limited by the pinhole camera model and lack effective modeling of defocus effects. Departing from this, we introduce DOF-GS — a new 3DGS-based framework with a finite-aperture camera model and explicit, differentiable defocus rendering, enabling it to function as a post-capture control tool. By training with multi-view images with moderate defocus blur, DOF-GS learns inherent camera characteristics and reconstructs sharp details of the underlying scene, particularly, enabling rendering of varying DOF effects through on-demand aperture and focal distance control, post-capture and optimization. Additionally, our framework extracts circle-of-confusion cues during optimization to identify in-focus regions in input views, enhancing the reconstructed 3D scene details. Experimental results demonstrate that DOF-GS supports post-capture refocusing, adjustable defocus and high-quality all-in-focus rendering, from multi-view images with uncalibrated defocus blur.

†: Denotes joint advising, co-corresponding authors.

1. Introduction

Despite the advancements made in digital photography, the need to set parameters like focus, aperture and exposure at capture-time remains a limitation, particularly for casual photographers facing dynamic conditions. Although methods such as light-field [6, 19, 27] and multi-/coded-aperture photography [4, 5, 7] offer some post-capture control, they typically require specialized hardware, limiting their accessibility, especially on mobile devices. Recent advancements in neural 3D reconstruction techniques, such as neural radiance fields (NeRF) [1, 17, 18] and 3D Gaussian Splatting (3DGS) [2, 9, 29] enable high-quality view synthesis from multi-view captures. Specifically, 3DGS allows for realistic post-capture viewpoint control in real-time, making it attractive for a wide range of applications, including immersive, interactive experiences like augmented and virtual reality (AR/VR). Beyond viewpoint control, however, adjusting parameters like focus and aperture post capture, and rendering their associated defocus and depth-of-field (DOF) effects, remain challenging in these approaches due to their inherent pinhole camera model assumption.

Some recent variants of these methods although model defocus blur, they are insufficient to support post-capture control of focal and aperture parameters due to their black-box

modeling. For instance, previous NeRF-based techniques [13, 15, 21] simulate defocus blur by estimating per-ray blur kernels and aggregating neighboring rays. However, these methods do not explicitly model a finite aperture camera, overlooking the relationships between point spread functions across pixels, limiting controllable defocus rendering, post-optimization. Moreover, these approaches are computationally expensive, with training durations ranging from hours to nearly a day [21], and slow novel view synthesis with speeds typically falling below 1 FPS even at moderate resolutions like 600×400 pixels [12]. Recently, 3DGS-based methods [12, 22] have attempted to model defocus blur with neural networks that predict per-Gaussian shape offsets or per-pixel blur kernels in image space. However, these techniques still lack built-in support for intuitive post-capture control through adjustable camera parameters.

In this paper, we introduce DOF-GS, a 3DGS-based framework that *incorporates a finite-aperture camera model* for modeling and rendering depth-of-field effects in reconstructed scenes, moving beyond the pinhole camera limitations in existing methods. By introducing a finite-aperture camera model, DOF-GS enables the adjustment of focal distance and aperture to render defocus effects, correctly representing depth-related blurring via the circle-of-confusion (CoC). Inspired by the surface-splatting based technique [10], our framework efficiently renders defocus effects by blurring individual Gaussians based on the learnable CoC cues, before compositing the final image via rasterization. Since Gaussian points optimized on sharp images do not naturally form defocus effects under this rendering scheme, we guide the model using defocus blur from captured images to enable smooth and realistic defocus rendering. To support optimization of Gaussian points and *learning the camera characteristics with defocused images*, we design the DOF rendering process to be *differentiable*. Additionally, to counteract reduced sharpness due to defocus blur in multi-view inputs, we enhance scene details by leveraging CoC cues emerging during optimization.

To summarize, our approach extends the 3DGS framework with three key enhancements: 1) **CoC-guided DOF rendering:** A novel rendering process integrated into the 3DGS rasterizer, allowing efficient and differentiable modeling of defocus effects. 2) **Learnable camera parameters:** Two adaptive parameters per view to dynamically fit defocus levels in training views and enable flexible DOF adjustments, post-optimization. 3) **CoC-guided detail enhancement:** A lightweight CoC-guided neural network that predicts in-focus mask for each view, which are utilized to enhance details within All-in-Focus renderings during optimization.

We experimentally test our framework on scenes from the multi-view defocus blur dataset [15]. Results demonstrate that our method effectively utilizes defocus blur from multi-view input defocused images, enabling on-demand DOF rendering and post-capture refocusing (Figure 1).

2. Related Work

Neural Radiance Field and 3D Gaussian Splatting. By bridging 2D multi-view images with the underlying 3D scene via differentiable volume rendering, NeRF [17] has made great progress in 3D reconstruction and novel view synthesis. In recent years, NeRF has been extended to handle dynamic scenes [14, 23] and enhanced with explicit representations for improved performance [1, 3, 11, 18]. Recently, Kerbl et al. [8] introduce the 3D Gaussian Splatting technique, achieving high-quality real-time renderings at notably high resolutions. Subsequent developments have further enhanced its capabilities by handling anti-aliasing [30], dynamic scenes [2, 29], and improving rendering speeds for large-scale environments [9]. However, most NeRF- and 3DGS-based approaches prioritize producing sharp images, often overlooking modeling and rendering of defocus blur.

Defocus Deblurring in NeRF and 3DGS. Some studies simulate defocus blur during NeRF or 3DGS training to address multi-view inconsistencies and blurry reconstructions caused by defocus blur in multi-view inputs. For example, NeRF-based methods [13, 15, 21] simulate defocus by predicting pixel-wise blur kernels per view and aggregating multiple rays at each pixel. However, these methods lack an explicit finite-aperture camera model, leading to flexible but arbitrarily predicted blur kernels without accounting for relationships among point spread functions across different pixels, thus limiting post-capture focus and aperture control. Moreover, the aggregation of multiple rays in volume rendering demands extensive sampling, making these methods highly computationally expensive. Recently, some 3DGS methods [12, 22] integrate defocus modeling to mitigate the effect of defocus blur and improve rendering efficiency. These methods use neural networks to predict per-Gaussian shape offsets or dense pixel-wise kernels in image space to fit the defocus observed in training views. However, due to this black-box modeling, they still lack inherent support for post-capture focus and aperture control.

Defocus Rendering in NeRF. Defocus blur is typically rendered by simulating image formation with a finite-aperture camera model. Existing methods like RawNeRF [16] and NeRFocus [26] synthesize DOF effects using NeRF, but assume all-in-focus (AiF) inputs. The closest NeRF-based counterpart to our approach, DoF-NeRF [28], also aims at controllable DOF rendering post-training but suffers from high computational costs of volume rendering, and differs from our method in defocus blur modeling and simulation. To address the limitations of the pinhole camera model in existing 3DGS methods, we extend the 3DGS framework to include a finite-aperture model with two learnable parameters: aperture size and focal distance. Our approach enables defocus rendering by applying defocus effects to individual Gaussians before compositing the final image, inspired by

surface-splatting techniques [10]. Notably different from [10], our rendering process is fully differentiable, enabling optimization of associated camera parameters—such as aperture diameter and focal distance—alongside the underlying 3D scene based on multi-view defocused images.

3. Method

Our approach, illustrated in Figure 3, reconstructs a 3D scene and enables post-capture depth-of-field (DOF) rendering from multi-view images $\{\mathbf{I}_m\}_{m=1}^{M}$ exhibiting varying and arbitrary defocus blur. Building on the 3DGS framework [8], we represent the scene with a set of anisotropic 3D Gaussian points $\{\mathcal{G}_k\}_{k=1}^{K}$, optimize camera parameters to allow flexible, post-capture adjustments, while also mitigating defocus blur effects for fine 3D scene reconstruction. In Section 3.1, we outline the scene representation, 3DGS rasterization-based rendering process, and our camera and physical models for depth-of-field. Section 3.2 details our differentiable DOF rendering, crucial for simulating defocus blur in training views. Finally, Section 3.3 describes an optimization strategy to enhance scene detail in reconstruction by using defocus cues derived from the DOF rendering.

3.1. Preliminaries

3.1.1. 3D Gaussian Splatting

The 3DGS technique [8] is a point-based method where 3D scenes are explicitly represented through an array of 3D Gaussian points given by:

$$\mathcal{G}_k(\mathbf{p}) = \exp\left(-\frac{1}{2}\left(\mathbf{p}-\mathbf{u}_k\right)^T \mathbf{\Sigma}_k^{-1}\left(\mathbf{p}-\mathbf{u}_k\right)\right), \quad (1)$$

where $\mathbf{p}$ denotes 3D points in world space. Specifically, each Gaussian is defined by following attributes: a center position $\mathbf{u}_k$, covariance matrix $\mathbf{\Sigma}_k$ derived from anisotropic scaling $\mathbf{s}_k$ and a quaternion vector $\mathbf{q}_k$, as well as opacity o_k and spherical harmonics coefficients $\mathbf{h}_k$. Ultimately, a scene is parameterized as $\mathcal{S} = \{\mathcal{G}_k : \mathbf{u}_k, \mathbf{q}_k, \mathbf{s}_k, o_k, \mathbf{h}_k\}_{k=1}^{K}$. This method realizes high-quality efficient rendering through a customized Gaussian rasterization process. Following the approach by Zwicker et al. [31], the 3D Gaussian points are first projected to 2D screen space, during which the Jacobian $\mathbf{J}$ of the projective transformation is calculated, and the covariance matrix $\mathbf{\Sigma}'_k$ of the 2D Gaussian after projection is computed as

$$\Sigma'_k = \mathbf{JW}\mathbf{\Sigma}_k\mathbf{W}^T\mathbf{J}^T. \quad (2)$$

$\mathbf{W}$ represents the view matrix transforming points from world space to camera space. The 2D Gaussian in screen space is formulated as:

$$\mathcal{G}'_k(\mathbf{x}) = \exp\left(-\frac{1}{2}\left(\mathbf{x}-\mathbf{u}'_k\right)^T \left(\mathbf{\Sigma}'_k\right)^{-1}\left(\mathbf{x}-\mathbf{u}'_k\right)\right), \quad (3)$$

where $\mathbf{u}'_k$ represents the 2D center position, post-projection. Although $\mathcal{G}'_k$ has an infinite support in theory, it is evaluated on a limited range with a cutoff radius t in practice. Therefore, each pixel $\mathbf{x}$ is associated with only a small number $\mathcal{N}_x$ of the Gaussians in the scene. Finally, the projected Gaussians are rendered with alpha blending:

$$\tilde{\mathbf{I}}(\mathbf{x}) = \sum_{i=1}^{\mathcal{N}_x} T_i \alpha_i \mathbf{c}_i, \quad \alpha_i = \mathcal{G}'_i(\mathbf{x}), \quad (4)$$

where $T_i = \prod_{j=1}^{i-1}(1-\alpha_j)$ is transmittance, $\mathbf{c}_i$ denotes the view-dependent color of i-th Gaussian associated with the queried pixel $\mathbf{x}$, $\tilde{\mathbf{I}}(\mathbf{x})$ is rendered color on pixel $\mathbf{x}$. As Equation (4) is fully differentiable, 3DGS reconstructs a 3D scene by comparing rendered views against training views:

$$\min_{\mathcal{S}} \sum_{m=1}^{M} \mathcal{L}_{\text{rec}}(\tilde{\mathbf{I}}_m, \mathbf{I}_m), \quad (5)$$

where $\mathcal{L}_{\text{rec}}$ measures errors between two images through a weighted combination of MSE and DSSIM loss: $\mathcal{L}_{\text{rec}}(\mathbf{I}_a, \mathbf{I}_b) = ||\mathbf{I}_a - \mathbf{I}_b|| + \lambda_{\text{dssim}}\mathcal{L}_{\text{dssim}}(\mathbf{I}_a, \mathbf{I}_b)$ and m is the index running over the training views.

3.1.2. Camera model for DOF rendering

We use a simplified thin lens model for rendering DOF. A camera model is typically specified by the following attributes: focal length of the lens F, aperture diameter A, and focal distance of the object f. As illustrated in Figure 2, the object points deviating from the focal plane appear out-of-focus, and forms the circle-of-confusion (CoC). The radius of CoC for each point is determined by the depth z_o of the object point and the camera attributes aforementioned, and is given by

$$R^{(\text{coc})} = \frac{1}{2} FA \frac{|z_o - f|}{z_o(f-F)}. \quad (6)$$

As the lens focal length F is often much smaller than the focal distance f and depths of object points, we simplify Equation (6) to

$$R^{(\text{coc})} = \frac{1}{2} Q \left|\frac{1}{z_o} - \frac{1}{f}\right|, \quad (7)$$

where $Q = FA$ is a simplified aperture parameter.

3.2. Differentiable DOF Rendering

To effectively leverage the guidance of natural defocus blur present in inputs, while also mitigating possible reconstruction artifacts caused by defocus blur (as shown in Figure 1), we develop a differentiable DOF rendering pipeline. Specifically, we assume that a blurred image can be generated by first applying depth-dependent blur to each Gaussian in the scene, followed by rendering the image from these blurred

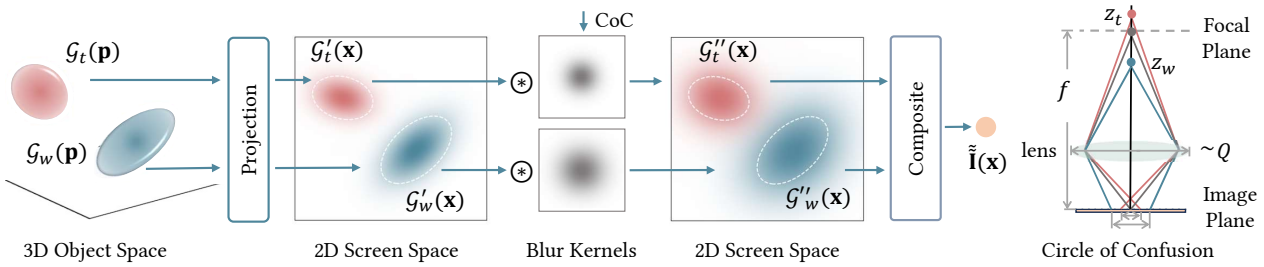


Figure 2. Illustration of the depth-of-field rendering and camera model. During the DOF rendering (left), 3D Gaussians $\mathcal{G}_k$ are projected to 2D screen space. Each projected 2D Gaussian $\mathcal{G}'_k$ is then convolved with a blur kernel, and the final color is composited from convolved Gaussians $\mathcal{G}''_k$. White dashed lines highlight convolution effects. The blur kernel for each 2D Gaussian is determined by the radius of its Circle-of-Confusion (CoC), which results from the finite-aperture camera model (right). With the adopted camera model with an aperture parameter Q, which is not a pinhole, object points deviating from the focal distance f form a region, known as the CoC, rather than a point.

Gaussians [10]. We illustrate the rendering process in Figure 2 and detail it below.

As illustrated in Figure 2, each 2D Gaussian $\mathcal{G}'_k$, representing the 2D projection of a 3D Gaussian point, is convolved with a Gaussian blur kernel proportional to the CoC determined by the camera model described in Section 3.1.2. We assume uniform depth across each 2D Gaussian's support, which is set as z_k, *i.e.*, the z-coordinate of the transformed center position in camera space. With the CoC radius $R_k^{(\text{coc})}$ calculated from Equation (6), we construct the blur kernel as:

$$\mathcal{G}_k^{(\text{coc})} = \frac{1}{2}\exp\left(\mathbf{x}^T\mathbf{\Sigma}_k^{(\text{coc})}\mathbf{x}\right),\ \mathbf{\Sigma}_k^{(\text{coc})} = a\mathbf{I} = \begin{bmatrix} a & 0 \\ 0 & a \end{bmatrix}. \quad (8)$$

a is a scalar, $\mathbf{I}$ is an identity matrix, and $\mathbf{\Sigma}_k^{(\text{coc})} = a\mathbf{I}$ means that $\mathcal{G}_k^{(\text{coc})}$ is isotropic. We determine a's value by solving

$$\min_a \left\| \frac{1}{\pi\left(R_k^{(\text{coc})}\right)^2} - \frac{1}{2\pi a}\exp\left(-\frac{1}{2}\frac{\mathbf{x}^T\mathbf{x}}{a}\right)\right\|_2, \quad (9)$$

which gives $a = \frac{1}{2ln4}\left(R_k^{(\text{coc})}\right)^2$, making $\mathcal{G}_k^{(\text{coc})}$ approximate a uniform intensity distribution within the CoC. Applying Gaussian kernels for blur convolution maintains the integrity of color composition, as convolving two Gaussians results in another Gaussian, preserving consistency with the original rasterization process. The convolved 2D Gaussian is defined by $\mathcal{G}''_k = \mathcal{G}'_k \circledast \mathcal{G}_k^{(\text{coc})}$, where $\circledast$ denotes convolution. Figure 2 illustrates the changes introduced by the convolution to individual 2D Gaussians. Since the convolution changes the associations between the 2D Gaussians and pixels, we reevaluate the number of Gaussians linked to each pixel, denoted as $\mathcal{N}'_x$. Subsequently, the color of each pixel $\mathbf{x}$ is obtained by accumulating contributions from these associated Gaussians. This process produces the output image $\tilde{\tilde{\mathbf{I}}}$, resulting in a refined optimization objective for scene reconstruction:

$$\min_{\{\mathcal{S},\,\{f_m,\,Q_m\}_{m=1}^M\}} \sum_{m=1}^{M} \mathcal{L}_{\text{rec}}(\tilde{\tilde{\mathbf{I}}}_m, \mathbf{I}_m). \quad (10)$$

As indicated in Equation (10), the optimizable parameters now include both the underlying 3D scene $\mathcal{S}$ and the camera parameters $\{f_m, Q_m\}_{m=1}^M$ for training views. To ensure the efficiency and differentiability, we incorporate Gaussian convolution and derivative calculations into the CUDA-customized rasterization process [8]. Further details on the derivations and the initialization strategy for the camera parameters can be found in the Supplementary Material.

Depth and CoC map rendering. As illustrated in Figure 3, the differentiable DOF rendering not only renders the color image $\tilde{\tilde{\mathbf{I}}}$ but also a depth map, expressed as:

$$\tilde{\tilde{\mathbf{D}}}(\mathbf{x}) = \sum_{i=1}^{\mathcal{N}'_x} T'_i\alpha'_i z_i,\ \ \alpha'_i = \mathcal{G}''_i(\mathbf{x}),\ \ T'_i = \prod_{j=1}^{i-1}(1-\alpha'_j), \quad (11)$$

where z_i is the z-coordinate of the transformed center position in camera space. Moreover, we also render a CoC map $\mathcal{M}^{(\text{coc})}$ as:

$$\mathcal{M}^{(\text{coc})}(\mathbf{x}) = \sum_{i=1}^{\mathcal{N}'_x} T'_i\alpha'_i R_i^{(\text{coc})},\ \ \alpha'_i = \mathcal{G}''_i(\mathbf{x}),\ \ T'_i = \prod_{j=1}^{i-1}(1-\alpha'_j). \quad (12)$$

Empirical observations indicate that the rendered CoC map $\mathcal{M}^{(\text{coc})}$ becomes highly correlated with the blur levels of pixels in training image as the optimization progresses, as shown in Figure 4.

3.3. CoC-Guided Detail Enhancement

The presence of large out-of-focus areas in training images can lead to the reconstructed scene prioritizing defocus blur effects, potentially compromising the scene detail clarity. To address this, we introduce a CoC-guided detail enhancement strategy that strengthens detail recovery by also supervising All-in-Focus images rendered by setting aperture to 0. Since explicit labels for in-focus regions are unavailable, we devise

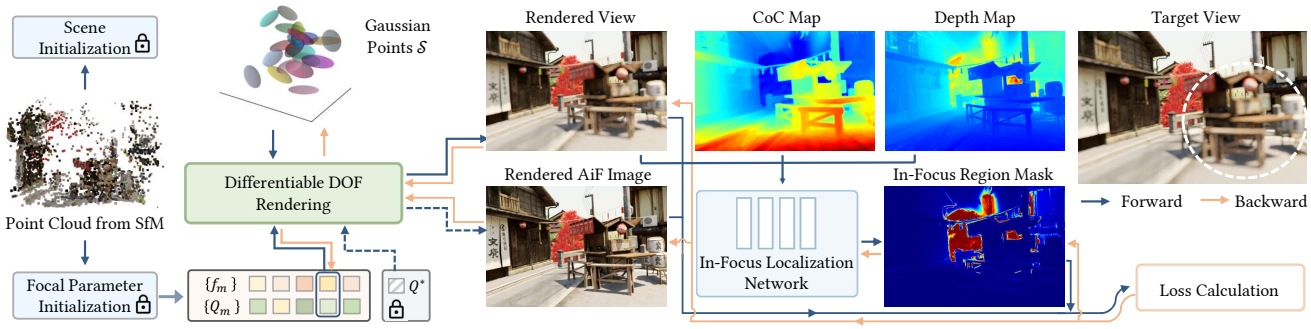


Figure 3. Pipeline of the proposed DOF-GS. We start with camera poses and an initial sparse point cloud estimated from defocused images. For each view, we introduce and initialize two learnable parameters: focal distance and aperture parameter. During optimization, for each sampled view m, we render a defocused image utilizing the DOF rendering scheme with camera parameters $\{f_m, Q_m\}$ to fit the target view. Meanwhile, we render an All-in-Focus (AiF) image with the fixed aperture parameter Q^*. To enhance scene details by appropriate AiF image supervision, we introduce an In-Focus Localization Network that utilizes the rendered CoC map and other cues to localize the in-focus regions within the target view. The underlying 3D scene, camera parameters, and network parameters are updated via backpropagation.

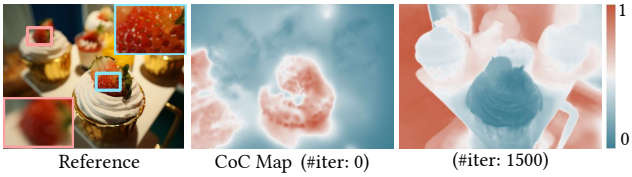


Figure 4. The rendered Circle-of-Confusion map quickly becomes correlated with pixel-wise blur levels in the reference view.

a lightweight neural module to estimate in-focus regions with the guidance of rendered CoC maps, which correlate with pixel-blurriness in training views. The neural module is called In-Focus Localization Network (ILN). Subsequently, we leverage in-focus masks output from ILN to supervise rendered All-in-Focus images.

In-Focus Localization Network. The In-Focus Localization Network is designed to guide the optimization to ensure proper focus on in-focus regions. We use rendered CoC map $\mathcal{M}_m^{(\text{coc})}$ as the primary input for the ILN to leverage the cues for per-pixel blur extent. We further augment the input with the rendered defocused image $\tilde{\tilde{\mathbf{I}}}_m$ and depth map $\tilde{\tilde{\mathbf{D}}}_m$ to provide contextual color and depth cues. Additionally, the training view index and pixel coordinates are also fed into the ILN through positional encoding, to assist in spatial localization. To minimize the computational overhead, we design the ILN to be lightweight, consisting of four 2D convolutional layers with a Sigmoid activation function. The ILN outputs a mask $\mathcal{M}_m^*$ that indicates the in-focus regions, which is given by

$$\mathcal{M}_m^* = \text{ILN}\left(\tilde{\tilde{\mathbf{I}}}_m, \tilde{\tilde{\mathbf{D}}}_m, \mathcal{M}_m^{(\text{coc})}, \text{PE}(m), \text{PE}(\mathbf{x})\right), \quad (13)$$

where $\text{PE}(\cdot)$ denotes positional encoding. As the ground truth masks for in-focus regions are not available, we regularize $\mathcal{M}_m^*$ using CoC map by a relaxed relative loss:

$$\mathcal{L}_{\text{mk}} = 1 - \frac{\text{Cov}(1-\mathcal{M}_m^*, \mathcal{M}_m^{(\text{coc})})}{\sqrt{\text{Var}(1-\mathcal{M}_m^*)\text{Var}(\mathcal{M}_m^{(\text{coc})})}}, \quad (14)$$

where the right term represents Pearson correlation that measures the distributional difference between the estimated in-focus region mask and the rendered CoC map, while disregarding scale inconsistencies. The loss $\mathcal{L}_{\text{mk}}$ guides $\mathcal{M}_m^*$ to have higher values for in-focus regions and lower values for out-of-focus regions.

Optimization Objectives. In each iteration, we render both a defocused image $\tilde{\tilde{\mathbf{I}}}_m$ and an All-in-Focus image $\tilde{\tilde{\mathbf{I}}}_m^*$ from the sampled viewpoint. The defocused image is rendered with the learnable aperture parameter Q_m and focal distance f_m, while the All-in-Focus image is rendered with a fixed aperture parameter $Q^* = 0$. With the estimated in-focus mask $\mathcal{M}_m^*$, we composite an intermediate image $\tilde{\tilde{\mathbf{I}}}_m^c$ by blending in-focus regions from $\tilde{\tilde{\mathbf{I}}}_m^*$ and out-of-focus regions from $\tilde{\tilde{\mathbf{I}}}_m$:

$$\tilde{\tilde{\mathbf{I}}}_m^c = \mathcal{M}_m^* \tilde{\tilde{\mathbf{I}}}_m^* + (1-\mathcal{M}_m^*)\tilde{\tilde{\mathbf{I}}}_m. \quad (15)$$

Then we calculate a loss $\mathcal{L}_{\text{detail}}$ by comparing the composited image against the training image:

$$\mathcal{L}_{\text{detail}} = ||\tilde{\tilde{\mathbf{I}}}_m^c - \mathbf{I}_m|| + \lambda_{\text{dssim}}\mathcal{L}_{\text{dssim}}(\tilde{\tilde{\mathbf{I}}}_m^c, \mathbf{I}_m). \quad (16)$$

This loss enables us to supervise All-in-Focus renderings with in-focused regions from input images. It allows optimizing masks $\mathcal{M}_m^*$ with gradients weighted by errors from both all-in-focus and defocused renderings, yielding smaller values for blurry pixels in input images. Meanwhile, this strategy is also beneficial in ensuring objects positions, especially in the transition regions, are correctly represented by achieving smooth transitions between the regions extracted from the rendered defocused image and the All-in-Focus



Figure 5. The CoC maps correlate with the depth-of-field (DOF) effects in the training views, while the predicted in-focus regions effectively locate the in-focus areas, blended with the reference image for better visualization. The CoC maps are normalized to the range $[0, 1]$.

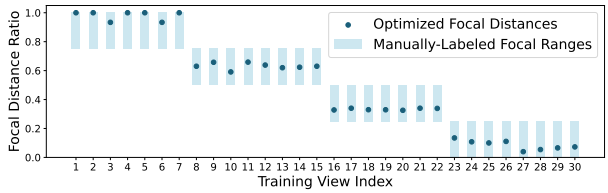


Figure 6. The focal distances optimized using our method closely align with the ranges labeled manually. Focal distances are converted to diopters and normalized for visualization.

image. Furthermore, as the errors in the rendered images decrease during the optimization process, we encourage the in-focus mask to become more binary by incorporating a regularizer:

$$L_{\text{reg}} = -\mathcal{M}_m^* \log\left(\mathcal{M}_m^*\right). \tag{17}$$

This regularizer encourages ILN to produce more distinct masks, focusing more on minimizing errors in in-focus regions during later iterations and simultaneously enhancing the quality of the All-in-Focus renderings. The overall optimization objective is now formulated as:

$$\min_{\{\mathcal{S},\, \Omega,\, \{f_m,\, Q_m\}_{m=1}^{M}\}} \mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{detail}} + \lambda_{\text{mk}}\mathcal{L}_{\text{mk}} + \lambda_{\text{reg}}\mathcal{L}_{\text{reg}}, \tag{18}$$

where Ω represents the parameters of the ILN, and λ_{mk} and λ_{reg} are weights for the loss terms $\mathcal{L}_{\text{mk}}$ and $\mathcal{L}_{\text{reg}}$, respectively.

4. Experiments and Analysis

This section covers our implementation (Section 4.1), examination of the role of the camera model in our framework (Section 4.2), results for novel view synthesis under the All-in-Focus setting (Section 4.3), and ablation study results (Section 4.4).

4.1. Implementation

The proposed method is implemented using a CUDA-customized rasterizer based on 3DGS [8] and the PyTorch library [20]. We optimize each scene for $40,000$ iterations. The balancing weights for loss terms are set to: $\lambda_{\text{dssim}} = 0.2$, $\lambda_{\text{mk}} = 0.001$, $\lambda_{\text{reg}} = 0.0001$. Further details on the optimization process, including an algorithmic table, are provided in the Supplementary Material.

We train and evaluate our approach using multi-view defocused images from the real and synthetic datasets provided by Ma et al.[15]. The scenes from the real dataset feature manually captured images, with a detailed scene list available in the Supplementary Material. For the synthetic scenes, to mimic real-world conditions where ground-truth camera poses are unavailable, we use camera poses and sparse point clouds estimated by COLMAP [24, 25] from the provided multi-view blurry rendered images.

4.2. Camera Model Analysis

In this section, we analyze the optimized focal distances for each training view, visualize the resulting circle-of-confusion (CoC) maps generated from the optimized camera parameters and the resultant 3D scene, and showcase visual outcomes from independent post-capture adjustments of both the focal distance and aperture size.

4.2.1. Focal Distance Analysis

To validate our method's ability to automatically adapt to training images captured at varying focal distances, we assess the consistency of the optimized focal distances with those in the training views. Since precise focal distance labels are unavailable, we sample three scenes and manually annotate a depth range for each view, approximating the camera's likely focus range during data capture. While it is difficult for humans to precisely determine focal distance from a single defocused image, an approximate range can be estimated, especially when comparing multiple images captured at different focal distances. Consequently, we partition the depth range of objects within each scene into up to four bins, and identify the most likely bin for the focal distance. Figure 6 shows the optimized focal distances and the annotated ranges, which align closely, thus validating the physical relevance of our camera model. This also validates the effectiveness of our framework in capturing defocus cues without auxiliary annotations. Additional visualizations can be found in the Supplementary Material.

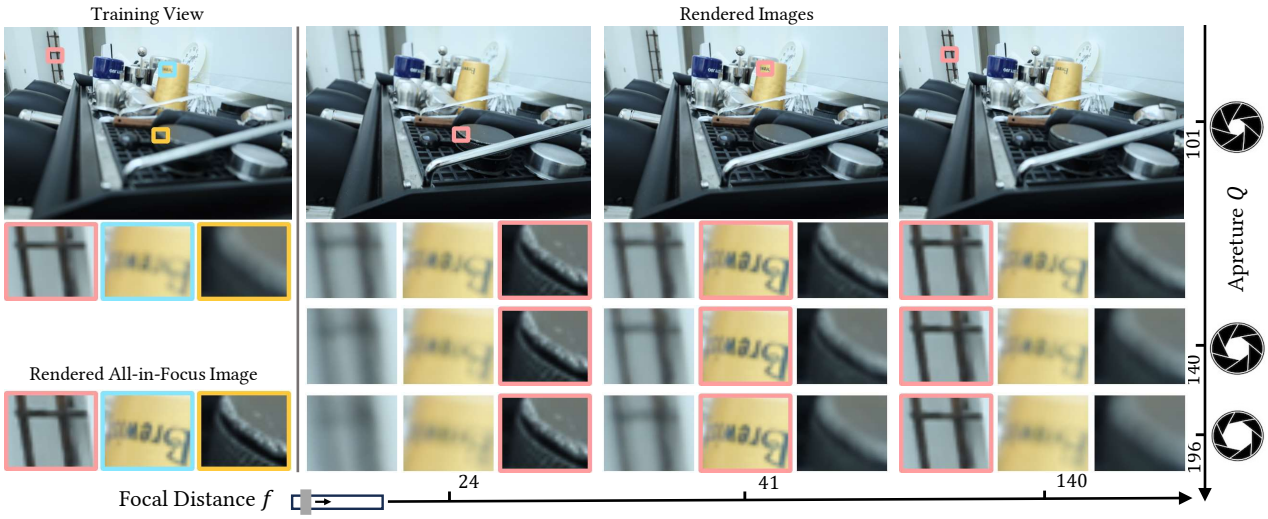


Figure 7. DOF rendering results with post-capture aperture and focal distance control. Adjusting focal distance mainly affects locations of out-of-focus and in-focus regions (highlighted in pink), while increasing aperture parameter makes out-of-focus regions increasingly blurry.

4.2.2. Effect of CoC Map and In-Focus Mask

Figure 5 presents rendered CoC maps, which exhibit a strong correlation with depth-related pixel blurriness, where higher values indicate greater blur. This further supports the physical relevance of our defocus rendering process and introduced camera model. Figure 5 also displays predicted masks that identify in-focus regions, which are crucial for improving All-in-Focus rendering details. As shown, these masks effectively locate the in-focus areas within training views.

4.2.3. Aperture and Focus Control

As discussed earlier, the introduced camera model and defocus rendering process are physically grounded. By effectively learning from the guidance of defocused images, we can simulate a wide range of depth-of-field effects through post-capture adjustments of focal distance and aperture settings. Figures 1 and 7 illustrate examples of these effects. As shown, with a fixed focal distance, the in-focus regions, such as the text region at $f = 41$ in Figure 7, remain sharp, while out-of-focus regions exhibit varying blur. When the aperture is kept constant and the focal distance is varied, the locations of in-focus and out-of-focus regions shift accordingly. Overall, the blur across different pixels transitions smoothly across images with different focal adjustments, for example, see images for aperture parameter $Q = 140$. Additional examples can be found in the Supplementary Material. To further demonstrate that our framework supports continuous control over focal distance and aperture—critical for practical applications—we provide video results in the Supplementary Video. These video results show depth-of-field effects rendered from multiple scenes, showcasing continuous adjustments to aperture, focal distance, or both.

4.3. All-in-Focus Novel View Synthesis

In addition to enabling post-capture depth-of-field rendering, our framework can also generate sharp All-in-Focus images from reconstructed scenes. We evaluate its performance in novel view synthesis under the All-in-Focus setting ($Q^* = 0$) and compare it with baseline methods, including NeRF[17], Mip-Splatting [30], Deblur-NeRF [15], DP-NeRF [13], PDRF [21], Deblur-GS [12] and BAGS [22]. We used the released code for baseline methods for consistency.

Table 1. Numerical results on real and synthetic defocus dataset. We color each cell as best , second best and third best .

	NeRF	Mip-Splatting	Deblur-NeRF	DP-NeRF	PDRF	BAGS	Deblur-GS	Ours
Real Dataset (Res.: 600 × 400)								
PSNR	22.70	21.23	23.98	24.12	24.31	24.17	24.21	24.12
SSIM	.6789	.6292	.7388	.7451	.7559	.7664	.7631	.7667
Synthetic Dataset (Res.: 600 × 400)								
PSNR	25.74	23.73	28.02	28.91	29.96	28.89	29.43	29.39
SSIM	.7769	.7314	.8491	.8687	.8923	.8859	.8907	.8914

We report the average performance on real scenes and synthetic scenes in Table 1. Our method performs comparably to existing deblurring NeRF and 3DGS methods. The comparable performance may result from the stricter constraints of the two-parameter camera model's global DOF modeling, which reduces flexibility compared to per-pixel kernel estimation. Sampled visual results are provided in Figure 8. As shown, Mip-Splatting, like vanilla NeRF, struggles to reconstruct fine details due to significant defocus blur in the input images. By contrast, our approach consistently outperforms or matches the performance of state-of-the-art NeRF-based and 3DGS-based methods, recovering details in challenging areas such as text, fruit stacks, and wall patterns.



Figure 8. Visual results for All-in-Focus novel view synthesis. Enlarged regions show our method's ability to recover sharp details.

Table 2. Numerical results on real defocus dataset at **higher** resolutions. We color each cell as best and second best

Method	Res.: 1200 × 800			Res.: 2400 × 1600		
	Avg. PSNR ↑	Avg. SSIM ↑	GPU Mem. Usage (GB) ↓	Avg. PSNR ↑	Avg. SSIM ↑	GPU Mem. Usage (GB) ↓
Mip-Splatting	20.51	.5995	8.75	20.35	.6220	32.13
BAGS	23.60	.7345	21.69	-	-	-
w/o. detail enhance	23.36	.7306	9.56	23.07	.7190	38.89
Full Model	23.66	.7376	13.06	23.45	.7221	39.05

-: Results are unavailable due to excessive memory required for training. The 3rd and 4th rows show results from two variants of our approach.

This improvement is due to our DOF rendering, which maintains differentiablity and effectively models defocus blur, thereby reducing its impact on scene reconstruction. Comprehensive quantitative comparisons and additional visual results are available in the Supplementary Material.

Performance with Increasing Resolution. To evaluate our method's adaptability to high-resolution inputs, we test it at resolutions of 1200×800 and 2400×1600. Table 2 summarizes peak GPU memory usage and performance comparisons with two other 3DGS-based methods, Mip-Splatting [30] and BAGS [22]. Our method performs robustly at both resolutions, with only a modest memory increase compared to Mip-Splatting. While the deblurring-focussed BAGS achieves similar quality at 1200×800, it demands 166% more memory. The memory efficiency of our approach is attributed to our defocus blur modeling, which mathematically applies depth-based Gaussian blurs directly to Gaussian points. Additionally, our variant, *Ours (w/o. detail enhance)*, conserves memory by disabling the All-in-Focus enhancement at a slight performance cost. These results highlight our method's strong adaptability to high-resolution inputs.

4.4. Ablation Study

In ablation studies, we evaluate the quantitative performance of All-in-Focus novel view synthesis for several variants. Results in Table 3 indicate that our detail enhancement strategy via imposing supervision on rendered All-in-Focus images significantly enhances PSNR and SSIM scores, confirming its effectiveness. Supplementary visual examples further illustrate improved details and reduced artifacts. The variants *Ours* (w/o. $\mathcal{L}_{\text{mk}}$) and *Ours* (w/o. $\mathcal{L}_{\text{reg}}$) achieve similar PSNR scores to the full model but show declines in SSIM, suggesting the positive impact of regularizing in-focus masks utilizing CoC maps and the binary regularizer $\mathcal{L}_{\text{reg}}$, as evidenced by the visual results in the Supplementary Material.

Table 3. Ablation of DOF-GS on real defocus dataset.

Variant	Ablation Items			Average Scores	
	$\mathcal{L}_{\text{detail}}$	$\mathcal{L}_{\text{mk}}$	$\mathcal{L}_{\text{reg}}$	PSNR ↑	SSIM ↑
Ours (w/o. detail enhance)	✗	✗	✗	23.36	0.7306
Ours (w/o. $\mathcal{L}_{\text{mk}}$)	✓	✗	✓	24.09	0.7642
Ours (w/o. $\mathcal{L}_{\text{reg}}$)	✓	✓	✗	24.09	0.7640
Ours (Full Model)	✓	✓	✓	**24.12**	**0.7667**

5. Conclusion

We present DOF-GS, a 3D Gaussian Splatting framework that integrates a finite-aperture camera model to enable refocusing and depth-of-field effects at runtime. DOF-GS effectively reconstructs high-quality 3D scenes from uncalibrated, defocused multi-view inputs, and achieves superior all-in-focus novel view synthesis with minimal increase in GPU memory. In particular, our method supports post-capture adjustments to camera parameters such as aperture and focal distance, enhancing imaging capabilities. Recovering sharp details for regions that remain blurry in all views remains challenging. A future direction is to incorporate camera pose optimization during training, as this could improve camera pose accuracy and, consequently, scene reconstruction quality in the presence of defocus blur. We anticipate our work to spark further advances in flexible, high-fidelity and high-dimensional imaging, scene modeling and analysis.

Acknowledgment

This work was partially supported by the National Key R&D Program of China (Grant No. 2022ZD0160801) and NSF Grant 2107454. We sincerely thank Guangyuan Zhao and Yuanxing Duan for fruitful discussions.

References

[1] Zhiqin Chen, Thomas Funkhouser, Peter Hedman, and Andrea Tagliasacchi. Mobilenerf: Exploiting the polygon rasterization pipeline for efficient neural field rendering on mobile architectures. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16569–16578, 2023. 1, 2

[2] Yuanxing Duan, Fangyin Wei, Qiyu Dai, Yuhang He, Wenzheng Chen, and Baoquan Chen. 4d gaussian splatting: Towards efficient novel view synthesis for dynamic scenes, 2024. 1, 2

[3] Sara Fridovich-Keil, Alex Yu, Matthew Tancik, Qinhong Chen, Benjamin Recht, and Angjoo Kanazawa. Plenoxels: Radiance fields without neural networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5501–5510, 2022. 2

[4] Paul Green, Wenyang Sun, Wojciech Matusik, and Frédo Durand. Multi-aperture photography. page 68–es, 2007. 1

[5] Shuji Habuchi, Keita Takahashi, Chihiro Tsutake, Toshiaki Fujii, and Hajime Nagahara. Time-efficient light-field acquisition using coded aperture and events. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 24923–24933, 2024. 1

[6] Chao-Tsung Huang, Yu-Wen Wang, Li-Ren Huang, Jui Chin, and Liang-Gee Chen. Fast physically-correct refocusing for sparse light fields using block-based multi-rate view interpolation. *IEEE Transactions on Image Processing*, PP:1–1, 11 2016. 1

[7] Yasutaka Inagaki, Yuto Kobayashi, Keita Takahashi, Toshiaki Fujii, and Hajime Nagahara. Learning to capture light fields through a coded aperture camera. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 418–434, 2018. 1

[8] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, and George Drettakis. 3d gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4), July 2023. 2, 3, 4, 6

[9] Bernhard Kerbl, Andreas Meuleman, Georgios Kopanas, Michael Wimmer, Alexandre Lanvin, and George Drettakis. A hierarchical 3d gaussian representation for real-time rendering of very large datasets. *ACM Transactions on Graphics*, 43(4), 2024. 1, 2

[10] J. Krivanek, J. Zara, and K. Bouatouch. Fast depth of field rendering with surface splatting. In *Proceedings Computer Graphics International 2003*, pages 196–201, 2003. 2, 3, 4

[11] Jonas Kulhanek and Torsten Sattler. Tetra-nerf: Representing neural radiance fields using tetrahedra. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 18458–18469, 2023. 2

[12] Byeonghyeon Lee, Howoong Lee, Xiangyu Sun, Usman Ali, and Eunbyung Park. Deblurring 3d gaussian splatting. *arXiv preprint arXiv:2401.00834*, 2024. 2, 7

[13] Dogyoon Lee, Minhyeok Lee, Chajin Shin, and Sangyoun Lee. Dp-nerf: Deblurred neural radiance field with physical scene priors. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12386–12396, 2023. 2, 7

[14] Jia-Wei Liu, Yan-Pei Cao, Weijia Mao, Wenqiao Zhang, David Junhao Zhang, Jussi Keppo, Ying Shan, Xiaohu Qie, and Mike Zheng Shou. Devrf: Fast deformable voxel radiance fields for dynamic scenes. *Advances in Neural Information Processing Systems*, 35:36762–36775, 2022. 2

[15] Li Ma, Xiaoyu Li, Jing Liao, Qi Zhang, Xuan Wang, Jue Wang, and Pedro V Sander. Deblur-nerf: Neural radiance fields from blurry images. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 12861–12870, 2022. 2, 6, 7

[16] Ben Mildenhall, Peter Hedman, Ricardo Martin-Brualla, Pratul P. Srinivasan, and Jonathan T. Barron. NeRF in the dark: High dynamic range view synthesis from noisy raw images. *CVPR*, 2022. 2

[17] Ben Mildenhall, Pratul P Srinivasan, Matthew Tancik, Jonathan T Barron, Ravi Ramamoorthi, and Ren Ng. Nerf: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision*, pages 405–421, 2020. 1, 2, 7

[18] Thomas Müller, Alex Evans, Christoph Schied, and Alexander Keller. Instant neural graphics primitives with a multiresolution hash encoding. *ACM transactions on graphics (TOG)*, 41(4):1–15, 2022. 1, 2

[19] Ren Ng, Marc Levoy, Mathieu Brédif, Gene Duval, Mark Horowitz, and Pat Hanrahan. Light field photography with a hand-held plenopic camera. *Technical Report CTSR 2005-02*, CTSR, 01 2005. 1

[20] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, Alban Desmaison, Andreas Kopf, Edward Yang, Zachary DeVito, Martin Raison, Alykhan Tejani, Sasank Chilamkurthy, Benoit Steiner, Lu Fang, Junjie Bai, and Soumith Chintala. Pytorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, pages 8024–8035. 2019. 6

[21] Cheng Peng and Rama Chellappa. Pdrf: Progressively deblurring radiance field for fast and robust scene reconstruction from blurry images. In *The 37th AAAI Conference on Artificial Intelligence*, 2023. 2, 7

[22] Cheng Peng, Yutao Tang, Yifan Zhou, Nengyu Wang, Xijun Liu, Deming Li, and Rama Chellappa. Bags: Blur agnostic gaussian splatting through multi-scale kernel modeling. *arXiv preprint arXiv:2403.04926*, 2024. 2, 7, 8

[23] Albert Pumarola, Enric Corona, Gerard Pons-Moll, and Francesc Moreno-Noguer. D-NeRF: Neural Radiance Fields for Dynamic Scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020. 2

[24] Johannes L Schonberger and Jan-Michael Frahm. Structure-from-motion revisited. In *Proceedings of the IEEE confer-*

ence on computer vision and pattern recognition, pages 4104–4113, 2016. 6

[25] Johannes L Schönberger, Enliang Zheng, Jan-Michael Frahm, and Marc Pollefeys. Pixelwise view selection for unstructured multi-view stereo. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11-14, 2016, Proceedings, Part III 14*, pages 501–518. Springer, 2016. 6

[26] Yinhuai Wang, Shuzhou Yang, Yujie Hu, and Jian Zhang. Nerfocus: neural radiance field for 3d synthetic defocus. *arXiv preprint arXiv:2203.05189*, 2022. 2

[27] Li-Yi Wei, Chia-Kai Liang, Graham Myhre, Colvin Pitts, and Kurt Akeley. Improving light field camera sample design with irregularity and aberration. *ACM Trans. Graph.*, 34(4), July 2015. 1

[28] Zijin Wu, Xingyi Li, Juewen Peng, Hao Lu, Zhiguo Cao, and Weicai Zhong. Dof-nerf: Depth-of-field meets neural radiance fields. In *Proceedings of the 30th ACM International Conference on Multimedia*, page 1718–1729, 2022. 2

[29] Zeyu Yang, Hongye Yang, Zijie Pan, and Li Zhang. Real-time photorealistic dynamic scene representation and rendering with 4d gaussian splatting. 2024. 1, 2

[30] Zehao Yu, Anpei Chen, Binbin Huang, Torsten Sattler, and Andreas Geiger. Mip-splatting: Alias-free 3d gaussian splatting. *Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. 2, 7, 8

[31] Matthias Zwicker, Hanspeter Pfister, Jeroen Van Baar, and Markus Gross. Ewa volume splatting. In *Proceedings Visualization, 2001. VIS'01.*, pages 29–538. IEEE, 2001. 3